

MODEL.PY

transformer :: construction vs forward flow

Two independent call sequences through the same six classes. Construction happens once, wires up submodules and their weights. Forward happens every batch, moves tensors through the already-built graph.

construction-time (build_transformer)

forward-time (encode / decode / project)

01 construction time

Runs once, when you call `build_transformer(...)`. No tensors exist yet — only nn.Module objects being created and nested inside each other.

○ build_transformer(...)

src_vocab_size

tgt_vocab_size

src_seq_len

tgt_seq_len

d_model=512

N=6

h=8

dropout=0.1

d_ff=2048

○ InputEmbeddings(d_model, src_vocab_size) → src_embed

InputEmbeddings(d_model, tgt_vocab_size) → tgt_embed

○ PositionalEmbedding(d_model, src_seq_len, dropout) → src_pos

PositionalEmbedding(d_model, tgt_seq_len, dropout) → tgt_pos

○ for _ in range(N): build one EncoderBlock per iteration, fresh weights each time

MultiHeadAttention(d_model, h, dropout) → self_attn

FeedForwardBlock(d_model, d_ff, dropout) → ff

EncoderBlock(self_attn, ff, dropout) → append to encoder_blocks

- `for _ in range(N):` build one `DecoderBlock` per iteration — needs two independent attention modules

```
MultiHeadAttention(d_model, h, dropout) → self_attn  
MultiHeadAttention(d_model, h, dropout) → cross_attn  
FeedForwardBlock(d_model, d_ff, dropout) → ff
```

```
DecoderBlock(self_attn, cross_attn, ff, dropout) → append to  
decoder_blocks
```

- `Encoder(nn.ModuleList(encoder_blocks))` → **encoder**
stores the list as `self.layers`, creates its own final `LayerNormalization`

- `Decoder(nn.ModuleList(decoder_blocks))` → **decoder**
same pattern — stores layers, creates final norm

- `ProjectionLayer(d_model, tgt_vocab_size)` → **projection_layer**

- `Transformer(encoder, decoder, src_embed, tgt_embed, src_pos, tgt_pos, projection_layer)`
just stores references to everything above — no computation, no `forward()`

- `for p in transformer.parameters(): xavier_uniform_(p) if p.dim() > 1`
recursively touches every weight nested inside every submodule above

02 forward time

Runs on every batch, once the object graph above already exists. `Transformer` has no single `forward()` — a training loop calls `encode`, then `decode`, then `project` as three explicit steps.

encode branch

○ **transformer.encode**(src, src_mask)
src: (batch, src_seq_len) long

○ **src_embed**(src)
→ (batch, seq, d_model)

○ **src_pos**(src)
→ (batch, seq, d_model)

○ **encoder**(x, mask): for layer in self.layers → x = layer(x, mask), N times, sequentially

residual[0](x, self_attn(x,x,x,src_mask))

residual[1](x, feed_forward(x))

○ **self.norm**(x) → **encoder_output**
(batch, src_seq, d_model) – computed once

decode branch

○ **transformer.decode**(encoder_output, src_mask, tgt, tgt_mask)
tgt: (batch, tgt_seq_len) long

○ **tgt_embed**(tgt) → **tgt_pos**(tgt)
→ (batch, seq, d_model)

○ **decoder**(x, enc_out, ...): N iterations – encoder_output is the same tensor every time

residual[0](x, **self_attn**(x,x,x,tgt_mask))

residual[1](x, **cross_attn**(x, enc_out, enc_out, src_mask))

residual[2](x, **feed_forward**(x))

○ **self.norm**(x) → **decoder_output**
(batch, tgt_seq, d_model)

↓ both branches feed into project – encode's output only ever re-enters via cross_attn above
↓

○ **transformer.project**(decoder_output)
projection_layer: Linear(d_model → tgt_vocab_size), then log_softmax(dim=-1)
→ (batch, tgt_seq, tgt_vocab_size)